



RAG Pipeline

*AICB-P1 · Ngày 14 · Đo lường chất lượng AI
một cách khoa học*

Ngô Thanh Tùng

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

*"Sếp hỏi: AI agent của mình
tốt hơn ChatGPT bao nhiêu?
Bạn nói sao nếu không có
benchmark?"*



Giữ câu hỏi này trong đầu khi học bài hôm nay

1. The Mathematics of Evaluation (Traditional vs. Generative)
2. RAG Evaluation Algorithms (Deep Dive)
3. Evaluating Multi-Agent Systems & Tool Calling
4. The Science of LLM-as-a-Judge & SOTA Benchmarks
5. MLOps, Cost, and Continuous Improvement
6. Lab 14

- Hiểu vì sao evaluation là engineering discipline, không phải cảm tính
- Thiết kế benchmark với golden dataset, edge cases, và stratified sampling
- Sử dụng LLM-as-Judge để đánh giá output tự động với rubric và scale rõ ràng
- Chạy RAGAS metrics: faithfulness, answer relevancy, context recall, context precision
- Thực hiện failure analysis và xây dựng continuous improvement loop

Evaluation report cho agent gồm benchmark 20 test cases, RAGAS scores, LLM-as-Judge results, failure analysis, và improvement recommendations

- 1 golden dataset: 20 question-answer pairs với expected answers
- 1 RAGAS evaluation: faithfulness, answer relevancy, context scores
- 1 LLM-as-Judge scoring: rubric 1–5 cho ít nhất 10 responses
- 1 failure analysis: 3 worst cases với root cause và fix recommendations

The Mathematics of Evaluation (Traditional vs. Generative)

From deterministic token-matching to probabilistic vector spaces. Discover the mathematical proofs behind why legacy NLP metrics fail in the generative era, and learn how to rigorously quantify semantic meaning.



1.1 The Deterministic Baseline

Discover the mathematical proofs behind why legacy NLP metrics fail in the generative era, and learn how to rigorously quantify semantic meaning.

ML Truyền thống (Tất định)

Các tác vụ như phân loại hoặc trích xuất thực thể có đầu ra cố định, nhị phân (VD: 1 hoặc 0, Đúng hoặc Sai). **Khớp chính xác (Exact Match)** là tiêu chuẩn vàng.

Generative AI (Xác suất)

Đầu ra là một phân phối xác suất trên toàn bộ không gian từ vựng.

Vấn đề cốt lõi

Có hàng ngàn cách diễn đạt hoàn toàn hợp lệ để trình bày cùng một ý nghĩa.

Bài học rút ra:

Khi AI tiến hóa thành một cỗ máy sinh văn bản, các thước đo được thiết kế cho các hệ thống phân loại hoàn toàn sụp đổ.

Trong các tác vụ dự đoán truyền thống — như xây dựng mô hình LightGBM để dự đoán khách hàng rời bỏ ngân hàng — việc đánh giá có cấu trúc rất chặt chẽ. Khách hàng hoặc là rời bỏ (1) hoặc không (0).

Precision & Recall

Precision (Độ chính xác): Trong số tất cả các dự đoán positive, có bao nhiêu dự đoán thực sự đúng?

Recall (Độ phủ): Trong số tất cả các case positive thực tế, mô hình đã bắt thành công được bao nhiêu trường hợp?

PREDICTED CLASS			
ACTUAL CLASS		TRUE POSITIVE (TP)	FALSE POSITIVE (FP)
		FALSE NEGATIVE (FN)	TRUE NEGATIVE (TN)

Điểm **F1** cân bằng giữa Precision và Recall bằng trung bình điều hòa:

$$2 \cdot (Precision \cdot Recall) / (Precision + Recall)$$

Sự thất bại của AI tạo sinh:

- Ground Truth (Nhãn chuẩn): "Hà **Nội** là **thủ** **đô**."
- Agent Output: "**Thủ** **đô** **của** **Việt** **Nam** là Hà **Nội**."

Kết quả:

Exact Match bằng 0. Điểm F1 cấp độ token tụt dốc không phanh, mặc dù ý nghĩa ngữ nghĩa hoàn toàn chính xác 100%.

Bài học rút ra:

Các phương pháp đánh giá truyền thống trừng phạt sự biến đổi phong cách, tạo ra một "cạm bẫy" nơi các câu trả lời sinh văn bản hoàn hảo bị đánh dấu là thất bại.

1.2 The Flaws of N-gram Matching

The illusion of word overlap: Deconstructing BLEU and ROUGE to understand why N-gram metrics punish natural paraphrasing while completely missing semantic contradictions.

- **BLEU (Bilingual Evaluation Understudy):**

Ban đầu được thiết kế cho dịch máy. Nó đo lường xem đầu ra của máy có chia sẻ cùng các n-grams (chuỗi từ) với câu tham chiếu của người hay không.

- **Trọng tâm:** Thiên lệch nặng nề về Precision. (Máy có tự bịa ra những từ không nên có ở đó không?)

- **Brevity Penalty (BP - Hình phạt độ ngắn):** Một hình phạt toán học được áp dụng nếu câu của máy ngắn hơn đáng kể so với câu tham chiếu, nhằm ngăn chặn việc mô hình gian lận bằng cách chỉ xuất ra một từ đúng duy nhất.

BLEU Score Calculation

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** Được thiết kế cho tác vụ tóm tắt văn bản.

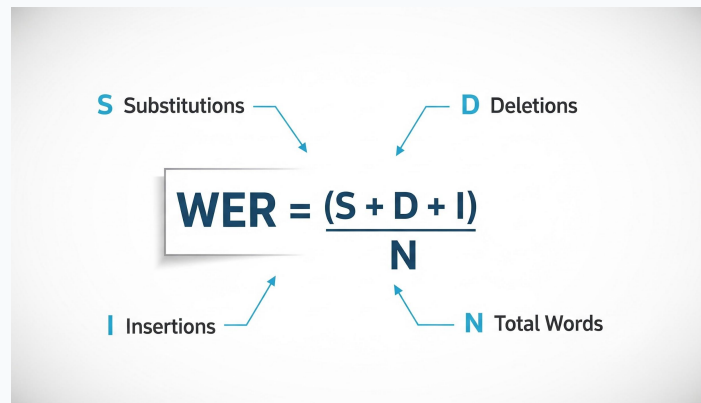
- **Trọng tâm:** Thiên lệch nặng nề về Recall. (Máy có nắm bắt được tất cả các khái niệm chính từ câu tham chiếu không?)

Phân Loại ROUGE

- **ROUGE-N:** Đo lường các cụm n-grams chồng chéo tiêu chuẩn.
- **ROUGE-L:** Dựa trên Chuỗi con chung dài nhất (Longest Common Subsequence - LCS).
- *Các từ không nhất thiết phải nằm liên tiếp nhau một cách nghiêm ngặt, chỉ cần đúng thứ tự tổng thể.*

- **WER (Tỷ lệ Lỗi Từ):** Ban đầu được thiết kế cho hệ thống Nhận dạng Giọng nói Tự động (ASR) và Dịch máy sơ khai. Nó đo lường "khoảng cách chỉnh sửa" (edit distance) nghiêm ngặt giữa câu của máy và câu tham chiếu.
- **Toán học nền tảng:** Dựa trên Khoảng cách Levenshtein.
- **Đặc điểm nghịch đảo:** Khác với BLEU hay ROUGE, điểm WER **càng thấp càng tốt** (0 là hoàn hảo). WER hoàn toàn có thể > 1.0 (vượt 100%) nếu LLM mắc lỗi sinh văn bản quá dài.
- **Điểm mù trong GenAI:** WER đòi hỏi một bản sao chép nguyên văn 1:1. Việc AI dùng từ đồng nghĩa hoặc đảo trật tự câu sẽ kích hoạt hàng loạt lỗi S và I.

Công thức tính WER


$$\text{WER} = \frac{S + D + I}{N}$$

The diagram illustrates the components of the WER formula. Arrows point from the following labels to their respective parts in the formula:

- S Substitutions** points to the 'S' in the numerator.
- D Deletions** points to the 'D' in the numerator.
- I Insertions** points to the 'I' in the numerator.
- N Total Words** points to the 'N' in the denominator.

Tại sao BLEU và ROUGE lại nguy hiểm đối với RAG và AI Agents?

Ví dụ minh họa

Ground Truth: "Agent **không bao giờ nên** hoàn tiền cho người dùng trong những điều kiện này."

AI Output: "Agent **nên** hoàn tiền cho người dùng trong những điều kiện này."

**Kết quả: Khớp chính xác 9/10 từ.
ROUGE chấm điểm này cực kỳ cao.**

Thực tế: Ngữ nghĩa hoàn toàn trái ngược nhau. Các thước đo N-gram hoàn toàn mù tịt trước sự phủ định logic.

Các thước đo N-gram chủ động trừng phạt các LLM thông minh sử dụng từ vựng đa dạng.

So sánh Agent

Câu tham chiếu: "Vị CEO đã mua một chiếc ô tô mới."

Agent A (Máy móc): "Vị CEO đã mua một chiếc ô tô mới." (**Điểm BLEU hoàn hảo**)

Agent B (Tự nhiên): "Ông chủ đã tậu một chiếc xe hơi mới." (**Điểm BLEU cực thấp**)

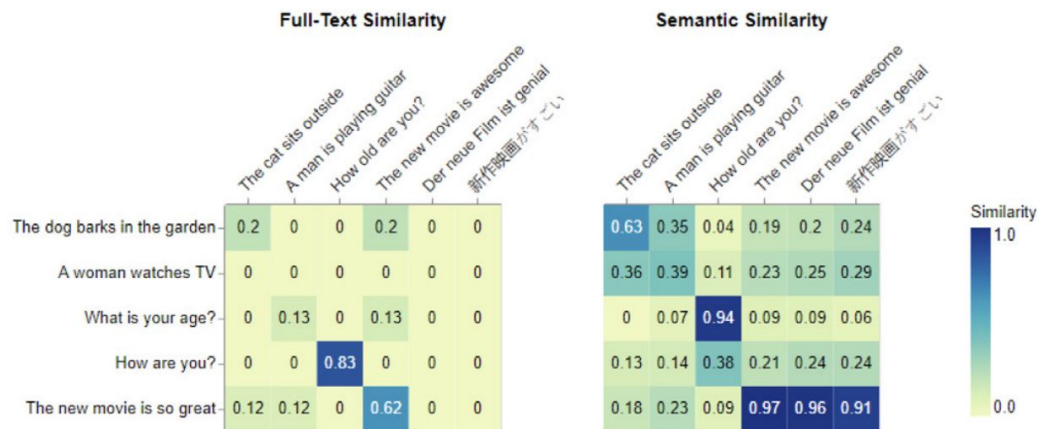
Bài học rút ra

Việc tối ưu hóa LLM theo BLEU/ROUGE ép mô hình phải sao chép-dán ngữ cảnh một cách vô hồn thay vì thực sự tổng hợp thông tin.

1.3 Transition To Semantic Similarity

Moving beyond lexical overlap: An introduction to vector embeddings and Cosine Similarity for evaluating the true semantic meaning of generated text.

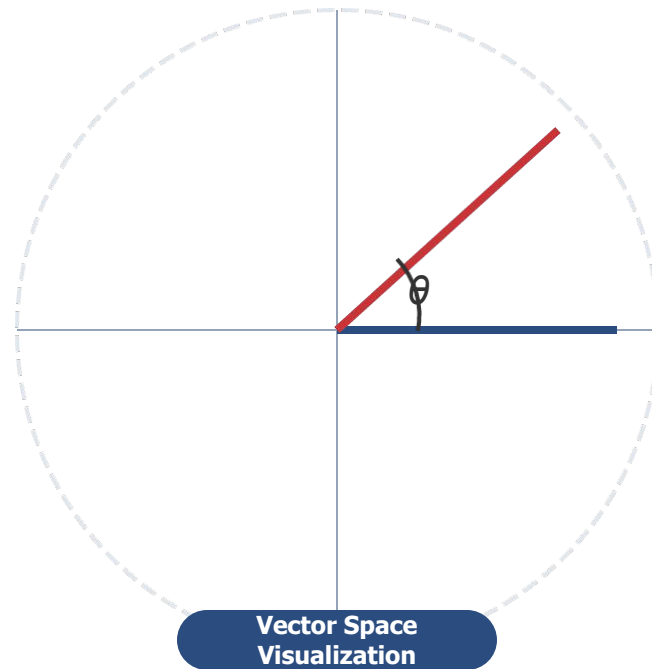
- Để thoát khỏi cạm bẫy từ vựng, chúng ta đánh giá dựa trên Ngữ nghĩa (Semantics).
- Chúng ta sử dụng một Embedding Model để ánh xạ các câu hoàn chỉnh vào một không gian toán học đa chiều R^d .
- Các câu có ý nghĩa giống hệt nhau sẽ được xếp gần nhau trong không gian vector này, ngay cả khi chúng không có chung bất kỳ từ vựng nào.



- Chúng ta đo khoảng cách giữa hai vector ngữ nghĩa bằng cách tính cosine của góc giữa chúng.

Công thức: $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$

- 1.0:** Các vector chỉ về cùng một hướng (Ý nghĩa gần như giống hệt nhau).
- 0.0:** Các vector vuông góc với nhau (Chủ đề hoàn toàn không liên quan).
- 1.0:** Các vector chỉ về hai hướng ngược nhau (Trái nghĩa trực tiếp, mặc dù hiếm gặp trong deep text embeddings).



- Cosine similarity giải quyết được vấn đề paraphrase, nhưng lại đưa ra một lỗ hổng mới: **Nó nắm bắt Chủ đề (Topic) tốt hơn là Sự thật (Fact).**

Câu 1: "Tôi thực sự yêu thích chiếc Apple Vision Pro mới."

Câu 2: "Tôi thực sự căm ghét chiếc Apple Vision Pro mới."

- **Vấn đề:** Vì chúng chia sẻ chung các đặc tính ngữ cảnh (review công nghệ, Apple, cảm xúc mạnh), Cosine Similarity của chúng thường **> 0.90**.
- **Bài học rút ra:** Các vector gặp khó khăn với những mâu thuẫn logic trực tiếp. Đó là lý do tại sao cuối cùng chúng ta cần đến **LLM-as-a-Judge**.

1.4 Statistical Rigor in Evaluation

Establishing the human baseline: Using Inter-Rater Reliability and Cohen's Kappa to mathematically quantify consensus before ever trusting an AI judge.

- Khi sử dụng hai chuyên gia để chấm điểm tập Golden Dataset, chúng ta tính toán hệ số Cohen's Kappa để tìm độ tin cậy thực sự.

$$\kappa = (p_o - p_e) / (1 - p_e)$$

- **p_o** : Tỷ lệ đồng thuận quan sát được (VD: cả hai chuyên gia cùng cho điểm 5 sao trong 70% trường hợp).
- **p_e** : Tỷ lệ đồng thuận kỳ vọng ngẫu nhiên (Xác suất thống kê họ đoán cùng một mức điểm).

(Lưu ý: Sử dụng hệ số Fleiss' Kappa nếu có từ 3 người chấm trở lên).

Thang điểm Kappa:

< 0.20 Đồng thuận rất yếu

0.41–0.60 Đồng thuận vừa phải (Rất phổ biến trong alignment LLM)

0.61–0.80 Đồng thuận đáng kể

0.81–1.00 Gần như hoàn hảo

Quy tắc vàng của AI Eval:

Nếu các chuyên gia con người khi chấm điểm benchmark chỉ đạt $\kappa = 0.55$, thì việc kỳ vọng Giám khảo AI tự động của bạn đạt độ đồng thuận 100% là điều không thể về mặt thống kê — và sai lầm về mặt khái niệm.

- Hình thức: Làm việc cá nhân
- Thời gian: 15 phút nghiên cứu + 10 phút thảo luận
- Nhiệm vụ của bạn:
 - 1. Tìm hiểu cơ chế hoạt động cốt lõi của 3 chỉ số quen thuộc: BLEU, ROUGE, WER.
 - 2. Nghiên cứu thêm 1 chỉ số mở rộng, ví dụ: METEOR.
 - 3. Tóm tắt trọng tâm (Focus), Ưu điểm (Pros), và Nhược điểm (Cons) của từng loại.

RAG Evaluation Algorithms (Deep Dive)

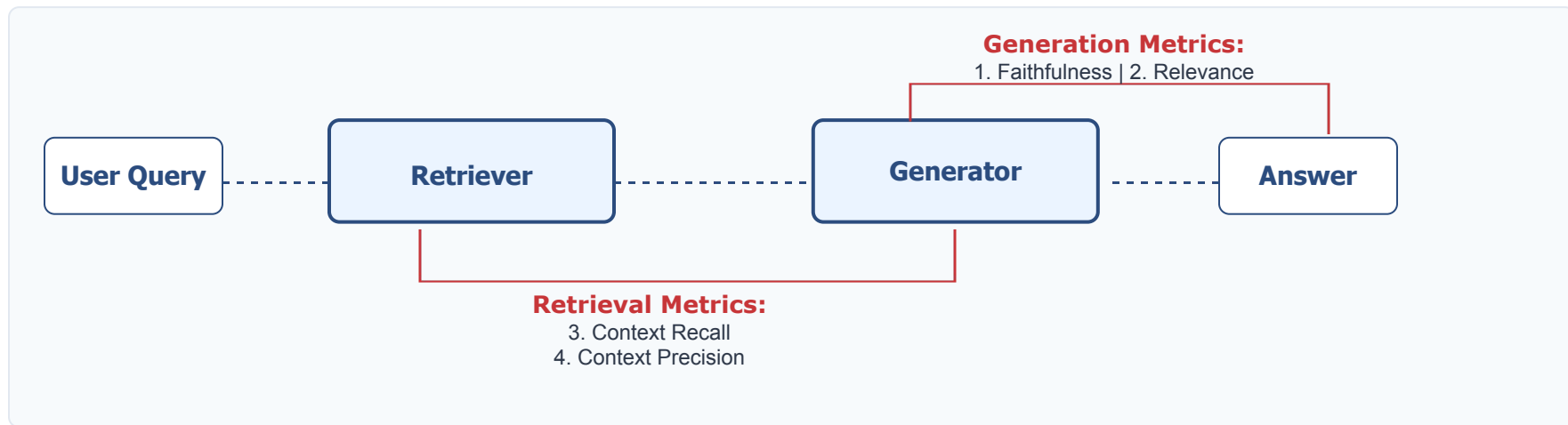
Unpacking the RAGAS framework: A deep dive into the mathematical algorithms—from Natural Language Inference to NDCG—that power modern, automated RAG evaluation.

2

2.1 The Evaluation Triad + Precision

The RAGAS framework mapped: Isolating system failures by deconstructing the pipeline into four independent metrics: Faithfulness, Relevance, Recall, and Precision.

- RAGAS (Retrieval Augmented Generation Assessment) đã trở thành tiêu chuẩn công nghiệp để đánh giá các hệ thống RAG mà không cần sự can thiệp của con người.
- Nó chia nhỏ pipeline RAG thành các thành phần riêng biệt và đánh giá chúng một cách độc lập.



- Hệ thống RAG thường thất bại theo hai hướng chính: Retriever không tìm thấy dữ liệu, hoặc LLM không tổng hợp được dữ liệu.
- RAGAS cô lập những thất bại này bằng toán học.
- Bạn có thể có một Generator hoàn hảo (Điểm: 1.0) ghép cặp với một Retriever cực kỳ tệ (Điểm: 0.2). Nếu LLM tóm tắt hoàn hảo các tài liệu bị sai, thì nó đang hoạt động chính xác theo đúng chức năng của mình.

Bài học rút ra: Đừng đánh giá RAG như một hộp đen duy nhất.

Search Engine

(Tìm kiếm)

Reasoning Engine

(Suy luận)

Hãy đánh giá chúng một cách riêng biệt.

2.2 Faithfulness / Hallucination Detection

The AI lie detector: Utilizing Natural Language Inference (NLI) to strictly quantify entailment and mathematically penalize hallucinations.

- **Định nghĩa:** Câu trả lời được tạo ra có dựa hoàn toàn vào ngữ cảnh được truy xuất không, hay LLM đã tự bịa ra thông tin?
- Việc đánh giá điều này đòi hỏi phải vượt ra ngoài Cosine similarity, vì một sự thật bịa đặt (hallucinated fact) vẫn có thể có độ tương đồng ngữ nghĩa rất cao với ngữ cảnh được truy xuất.

Giải pháp của RAGAS

Sử dụng một khái niệm NLP kinh điển: **Suy Luận Ngôn Ngữ Tự Nhiên (NLI)**.

NLI là nhiệm vụ xác định mối quan hệ logic giữa một **"Tiền đề"** (Premise - ngữ cảnh được truy xuất) và một **"Giả thuyết"** (Hypothesis - câu trả lời được tạo ra).

Ba trạng thái của NLI:

Entailment

(Kéo theo): Tiền đề hỗ trợ một cách nghiêm ngặt cho giả thuyết.

Contradiction

(Mâu thuẫn): Tiền đề trực tiếp bác bỏ giả thuyết.

Neutral

(Trung lập): Tiền đề không hỗ trợ cũng không bác bỏ giả thuyết (Đây là nơi xảy ra hầu hết các lỗi ảo giác của LLM!).

- Bạn không thể đánh giá một câu trả lời dài nhiều đoạn văn cho NLI cùng một lúc. Thuật toán phải chia nhỏ nó ra.
- Bước 1:** Sử dụng một LLM để phân tích câu trả lời và trích xuất ra một tập hợp các câu phát biểu độc lập (atomic statements): $S = \{s_1, s_2, \dots, s_n\}$.

CÂU TRẢ LỜI GỐC

"Albert Einstein sinh ra ở Đức và giành giải Nobel vào năm 1921."



CÁC CÂU PHÁT BIỂU TRÍCH XUẤT (ATOMIC)

s₁: "Albert Einstein sinh ra ở Đức."

s₂: "Albert Einstein giành giải Nobel vào năm 1921."

- **Bước 2:** Đối với mỗi câu phát biểu s_i , Judge LLM tính toán xác suất Entailment dựa trên ngữ cảnh được truy xuất: $P(\text{Entailment} \mid \text{Context}, s_i)$.
- Điểm số Faithfulness cuối cùng là tỷ lệ giữa các phát biểu được hỗ trợ hoàn toàn trên tổng số các phát biểu.

CÔNG THỨC & VÍ DỤ

$$\text{Faithfulness} = |V| / |S|$$

(trong đó $|V|$ là số lượng phát biểu được xác minh/entailed)

Ví dụ:

Nếu 4 trong số 5 phát biểu được hỗ trợ bởi ngữ cảnh, điểm Faithfulness là **0.8**.

2.3 Answer Relevance

Measuring intent alignment: Using reverse-engineering and vector math to ensure the AI actually addresses the prompt instead of being evasive.

- **Định nghĩa:** Câu trả lời cuối cùng có thực sự giải quyết truy vấn gốc của người dùng không, hay nó lan man, dài dòng và lảng tránh?
- Một mô hình có thể đạt Độ trung thực 100% (chỉ sử dụng ngữ cảnh truy xuất) nhưng lại có **Answer Relevance là 0.0** nếu Retriever kéo về những tài liệu không liên quan đến câu hỏi.
- **Vấn đề:** Chúng ta không thể tính toán trực tiếp Cosine Similarity giữa Câu hỏi và Câu trả lời vì cấu trúc ngữ pháp và ngữ nghĩa của chúng về cơ bản là khác nhau trong không gian vector.

- Để giải quyết sự không khớp trong không gian vector giữa Câu hỏi và Câu trả lời, **RAGAS sử dụng một phương pháp đảo ngược tuyệt vời.**
- Thay vì so sánh Truy vấn người dùng với Câu trả lời của Agent, nó **yêu cầu Judge LLM đọc Câu trả lời của Agent và đoán xem câu hỏi ban đầu là gì.**
- Chúng ta đang đánh giá xem **Câu trả lời "neo" chặt đến mức nào** vào ý định của Truy vấn gốc.

- **Bước 1:** Judge LLM tạo ra N câu hỏi giả định (VD: q_1, q_2, q_3) có thể đã dẫn đến câu trả lời đó.
- **Bước 2:** Nhúng (Embed) truy vấn gốc của người dùng (Q) và tất cả các câu hỏi giả định (q_i) vào không gian vector.
- **Bước 3:** Tính Cosine Similarity giữa Q và từng q_i , sau đó lấy giá trị trung bình.

Công thức:

$$\text{answer relevancy} = \frac{1}{N} \sum_{i=1}^N \cos(E_{g_i}, E_o)$$

2.4 Context Retrieval Metrics

Beyond basic recall: Utilizing advanced Information Retrieval (IR) math like MAP and NDCG to rigorously evaluate your vector database's ranking performance.

Bây giờ chúng ta chuyển từ việc đánh giá Generator sang đánh giá Retriever (Vector DB / BM25).

Context Recall

Độ phủ ngữ cảnh:

- Chúng ta có truy xuất được các thông tin cần thiết để trả lời câu hỏi không?
- Thường đo lường dựa trên Ground Truth.

Ranking Metrics

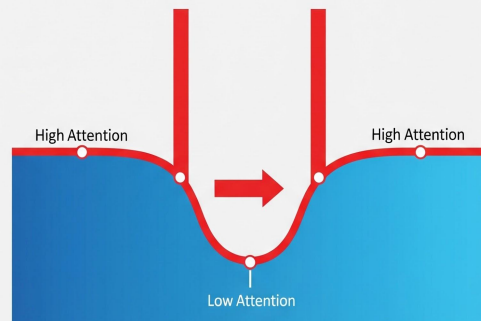
Tuy nhiên, chỉ trả về đúng tài liệu là chưa đủ.

Việc trả về tài liệu đúng cùng với 10 tài liệu không liên quan sẽ phá hủy hiệu năng của LLM.

Chúng ta cần các thước đo về thứ hạng (ranking metrics).

Tại sao thứ tự lại quan trọng?

- Các LLM thường mắc phải hiện tượng **"Lost in the Middle"**. Chúng dành sự chú ý rất cao cho phần đầu và phần cuối của context window, nhưng phớt lờ thông tin bị chôn vùi ở giữa.
- Nếu Retriever của bạn tìm thấy một tài liệu hoàn hảo nhưng xếp hạng nó là Chunk thứ 15 trên 20, LLM rất có thể sẽ thất bại trong việc sử dụng nó.
- Do đó, các thước đo đánh giá phải phạt nặng các Search Engine xếp hạng kém các tài liệu chính xác.



Mean Average Precision (MAP): Một thước đo Search engine cổ điển.

Nó đo lường Precision tại mọi điểm mà một tài liệu liên quan được tìm thấy trong danh sách đã được xếp hạng, sau đó tính trung bình.

Nếu bạn truy xuất 5 tài liệu và các tài liệu liên quan nằm ở hạng **#1 và #2**, MAP của bạn là hoàn hảo. Nếu chúng nằm ở hạng **#4 và #5**, MAP của bạn sẽ giảm mạnh.

MAP rất phù hợp cho đánh giá mức độ liên quan nhị phân (*tài liệu hoặc là có liên quan, hoặc là không*).

Cơ sở lý thuyết

- Trong RAG, mức độ liên quan hiếm khi là nhị phân. Một số chunk liên quan rất cao, một số lại chỉ liên quan một phần. Chúng ta sử dụng NDCG để đánh giá độ liên quan theo thang điểm.
- Cách thức hoạt động: Điểm số liên quan (rel_i) của một chunk được chia cho một hình phạt logarit dựa trên vị trí xếp hạng (i) của nó.
- Một chunk liên quan cao ở vị trí số 1 sẽ giữ nguyên toàn bộ giá trị. Ở vị trí số 10, mẫu số $\log_2(11)$ sẽ bóp nghẹt giá trị của nó.

Công thức DCG

$$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)} = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2(i+1)}$$

$$nDCG_p = \frac{DCG_p}{IDCG_p}$$

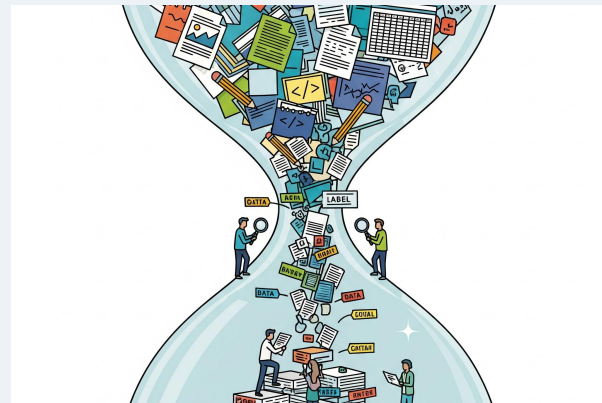
$$IDCG_p = \sum_{i=1}^{|REL_p|} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

2.5 Synthetic Test Data Generation

Breaking the human annotation bottleneck: Automatically generating high-quality, complex Golden Datasets using evolutionary algorithms like Evol-Instruct.

Thách thức về Ground Truth

- Để tính toán chính xác Context Recall và NDCG, bạn cần một tập dữ liệu Ground Truth.
- Vấn đề: Yêu cầu các chuyên gia trong ngành đọc thủ công PDF, viết các câu hỏi phức tạp và xác định các đoạn văn bản nguồn là công việc cực kỳ chậm chạp, tốn kém và không thể mở rộng (unscalable).
- Một hệ thống RAG trên dữ liệu doanh nghiệp có thể cần 1,000 cặp QA để đạt được độ tin cậy thống kê. Con người không thể đáp ứng được tốc độ này.



Giải pháp: Sử dụng LLM mạnh để tạo Golden Dataset

- Sử dụng các LLM mạnh (như GPT-4) để tự động tạo ra Golden Dataset.
- Để đảm bảo tập dữ liệu không chỉ toàn những câu hỏi đơn giản, chúng ta sử dụng Các thuật toán tiến hóa (như Evol-Instruct).
- Các quá trình Đột biến (Mutation):
 - **Làm sâu sắc (Deepening):** Lấy câu hỏi đơn giản và ép LLM phải viết lại nó để yêu cầu suy luận nhiều bước (multi-hop).
 - **Làm phức tạp (Complicating):** Thêm các ràng buộc (VD: "Hãy hỏi câu này như thể người dùng chỉ biết về tính năng X").

Quy trình 4 bước

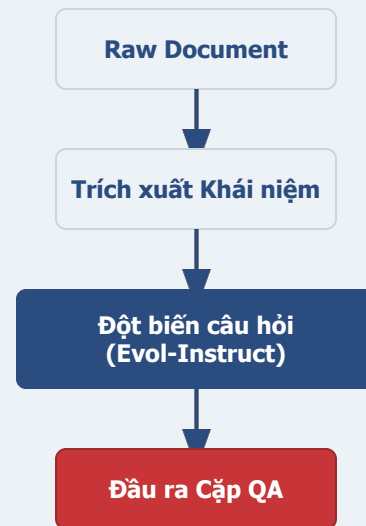
Bước 1 (Trích xuất Khái niệm): Đưa các raw chunks vào LLM. Yêu cầu nó trích xuất các thực thể và khái niệm chính.

Bước 2 (Tạo Câu hỏi): LLM tạo ra các câu hỏi chỉ dựa trên các khái niệm đó.

Bước 3 (Tạo Câu trả lời): Một LLM mạnh hơn tạo ra câu trả lời "Ground Truth".

Bước 4 (Sàng lọc): Một Critic LLM đánh giá cặp QA. Nếu câu hỏi có thể trả lời mà không cần tài liệu, hoặc quá mơ hồ, nó sẽ bị loại bỏ.

Sơ đồ Pipeline



Yêu cầu thực hành

- **Hình thức:** Làm việc cá nhân hoặc Theo cặp
- **Thời gian:** 30 - 45 phút
- **Nhiệm vụ của bạn:** Lựa chọn và lập trình (implement) ít nhất MỘT trong các kỹ thuật đánh giá sau cho hệ thống RAG: e.g. NLI, Reverse-Engineering
- **Nguồn Dữ liệu (Dataset):**
 - **Tái sử dụng (Khuyến nghị):** Lấy lại bộ dữ liệu Context - Question - Answer từ bài tập RAG của Ngày 8.
 - **Tạo mới:** Tự định nghĩa một file JSON/CSV nhỏ (khoảng 3-5 mẫu) tự tạo để kiểm thử nhanh.

Evaluating Multi-Agent Systems & Tool Calling

Transitioning from static pipelines to stateful orchestration: Rigorously evaluating agentic trajectories, tool-calling payload accuracy, and multi-turn conversational coherence.

3

3.1 The Multi-Agent State Space

Modeling autonomy: Transitioning from stateless RAG pipelines to stateful agents using Markov Decision Processes (MDPs) as the mathematical foundation for evaluation.

Đánh giá RAG giống như đánh giá công cụ tìm kiếm: Đầu vào → Truy xuất → Đầu ra. Nó là một pipeline phi trạng thái (stateless).

- Kiến trúc Multi-Agent (như LangGraph) mang tính trạng thái rất cao (stateful). Agent hành động, quan sát môi trường, cập nhật trạng thái và quyết định bước tiếp theo.
- Chúng ta không thể chỉ chấm điểm câu trả lời cuối cùng. Chúng ta phải chấm điểm cả hành trình mà agent đã đi qua.

Một agent có thể đưa ra câu trả lời chính xác, nhưng lại đốt mất \$5.00 tiền API và thực hiện 15 bước thừa thãi.



Để đánh giá agent khả thi, chúng ta ánh xạ chúng sang khái niệm Quá Trình Quyết Định Markov (MDP) trong Học tăng cường.

State (S - Trạng thái)

Context window hiện tại, bao gồm lịch sử hội thoại, vùng ghi nhớ (scratchpad) và các đầu ra công cụ trước đó.

Action (A - Hành động)

Tool mà agent chọn gọi, hoặc tin nhắn nó gửi cho người dùng.

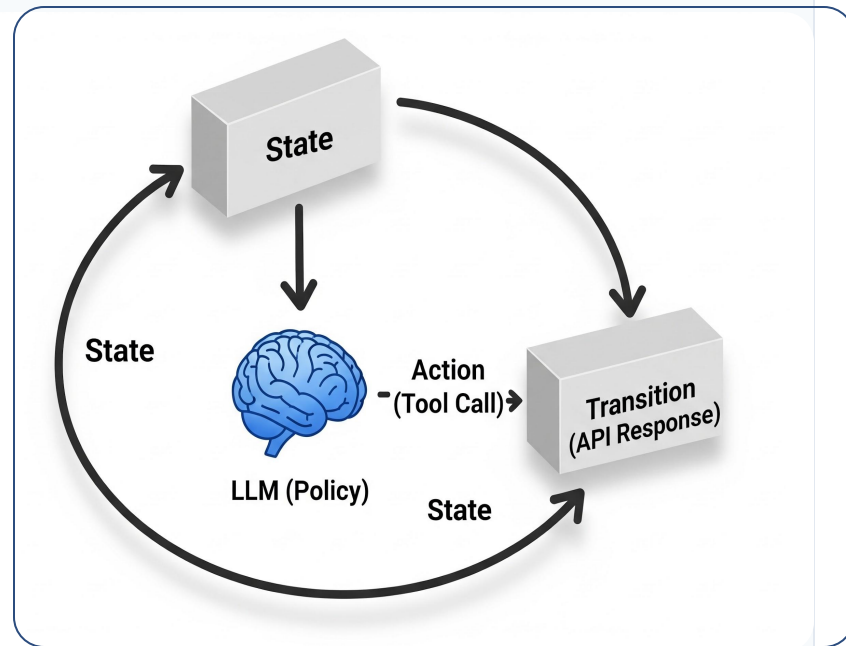
Transition (T - Chuyển đổi)

Cách môi trường thay đổi sau hành động (VD: API trả về JSON).

Reward (R - Phần thưởng)

Hành động đó có đưa agent đến gần hơn với mục tiêu không?

- Trong một MDP, "Policy" quyết định Action nào sẽ được thực hiện dựa trên State hiện tại.
- Trong các hệ thống Multi-Agent, LLM chính là Policy Engine.
- Đánh giá là đo lường chất lượng của Policy này: Đối mặt với một State phức tạp (hội thoại 5 lượt và một lệnh gọi API bị lỗi), LLM có biết cách khắc phục không, hay sinh ra ảo giác?



3.2 Trajectory Evaluation

Auditing the ReAct loop: Moving beyond final answers to strictly evaluate intermediate reasoning steps, trajectory efficiency, and redundant actions.

- Các Agent hoạt động dựa trên vòng lặp **Suy nghĩ** → **Hành động** → **Quan sát** (Kiến trúc ReAct).
- **Trajectory (Quỹ đạo)**: Toàn bộ chuỗi vòng lặp từ prompt ban đầu đến câu trả lời cuối cùng.
- Trajectory Evaluation đòi hỏi chúng ta đóng vai trò kiểm toán viên, duyệt qua các log và chấm điểm logic của các bước "Suy nghĩ" trung gian.
- "Planner Agent" có chia nhỏ prompt thành các tác vụ con chính xác trước khi chuyển cho "Researcher Agent" không?

- Định nghĩa: Agent đã mất bao nhiêu vòng lặp ReAct riêng biệt để giải quyết prompt?
- Hiệu quả là thước đo hàng đầu vì mỗi bước đều tiêu tốn token và làm tăng độ trễ (latency).

Tỷ lệ Quỹ đạo Tối ưu:

(Số bước tối thiểu để giải quyết tác vụ) / (Số bước thực tế agent đã thực hiện)

Ví dụ: Nếu một quy trình kéo dữ liệu tối ưu cần 2 lệnh gọi API, nhưng agent lóng ngóng gọi đến 6 lần, điểm hiệu quả là 0.33.

- Một trong những lỗi phổ biến nhất ở LLM agents là chứng **"hay quên"** hoặc thiếu tự tin.
- **Gọi thừa thãi:** Agent truy vấn một API (VD: `get_weather(Hanoi)`), nhận payload chính xác, nhưng sau đó lại gọi lặp lại chính công cụ đó với tham số giống hệt ở các bước sau.
- **Thước đo:** Trùng phạt bằng toán học đối với các quỹ đạo chứa các tổ hợp (`Action` + `Payload`) bị trùng lặp trong cùng một phiên.

- Các framework điều phối (như LangGraph) cho phép đồ thị chu trình (cyclic) — nghĩa là agent có thể kẹt trong các vòng lặp vô hạn.
- **Mô hình Deadlock:**
 1. Agent gọi API nhưng thiếu tham số.
 2. API trả về lỗi `400 Bad Request`.
 3. Agent nói "Tôi sẽ thử lại" và thực hiện chính xác lệnh gọi bị lỗi đó một lần nữa.
- Nếu không kiểm soát, điều này sẽ vắt kiệt ngân sách token chỉ trong vài phút.

- Đánh giá phải kiểm tra xem hệ thống xử lý các điểm bế tắc như thế nào.
- **Biện pháp 1 (Max Steps):** Hardcode giới hạn đệ quy trong bộ điều phối (ví dụ: `max_iterations=5`).
- **Biện pháp 2 (Decay Penalties):** Chèn các System Prompts làm tăng "chi phí" của việc lặp lại công cụ, ép policy engine cuối cùng phải chọn hành động "bỏ cuộc và hỏi lại người dùng".

3.3 Tool/Function Calling Accuracy

Validating the agent's actions: Rigorously evaluating tool selection accuracy and strict JSON payload adherence to prevent dangerous parameter hallucinations.

- Đánh giá Tool calling được chia thành hai thành phần toán học nghiêm ngặt. Đầu tiên là **Lựa chọn**.
- Độ chính xác Lựa chọn: Được xử lý như một bài toán **Phân loại Đa lớp** (Multi-Class Classification) cổ điển.
- Nếu bạn đưa cho agent 10 công cụ, nó có chọn đúng cái cần thiết không?

METRICS & RISK

Được đánh giá bằng **Precision** và **Recall**.

False Positives (gọi tool khi không nên gọi) thường nguy hiểm hơn nhiều so với **False Negatives** (bỏ sót không gọi tool).

- Việc chọn đúng tool sẽ vô dụng nếu các tham số bị sai.
- Độ chính xác **Tham số/Payload**: Đánh giá cấu trúc JSON chính xác được sinh ra bởi LLM.
- Chúng ta đánh giá đối chiếu với các định nghĩa **JSON schema** nghiêm ngặt (OpenAPI specs).

SCHEMA VALIDATION

LLM có xuất ra kiểu `integer` khi yêu cầu là `string` không?

Nó có bỏ sót một trường **required field** không?

- Chế độ lỗi nguy hiểm nhất của tool-calling là **ảo giác tham số** (parameter hallucination).
- Ví dụ: Người dùng yêu cầu "Hủy đơn hàng gần nhất của tôi." Agent chọn `cancel_order(order_id)`. Nhưng người dùng chưa từng cung cấp Order ID.
- Thay vì hỏi lại để làm rõ, LLM lại tự tin đoán mò:
`cancel_order(order_id="12345")`.

KIỂM TRA ĐÁNH GIÁ

Tập benchmark của bạn phải bao gồm các truy vấn bị thiếu thông tin.

Mục tiêu: Kiểm tra xem agent có kích hoạt hành động **"Yêu cầu làm rõ"** (Clarification Request) hay không thay vì sinh ảo giác tham số.

3.4 Conversational/Session Metrics

Evaluating the entire user session: Measuring multi-turn coherence, quantifying context decay, and tracking task resolution via automated Dialog State Tracking (DST).

Bây giờ chúng ta lùi góc nhìn ra toàn bộ Cuộc hội thoại.

Tính mạch lạc đa lượt (Multi-Turn Coherence): Agent có nhớ các ràng buộc được thiết lập từ đầu cuộc trò chuyện không?

SCENARIO MOCKUP

Lượt 1: "Tôi chỉ muốn chuyển bay dưới \$500."

Lượt 5: "Còn chuyển bay đi Tokyo thì sao?"

Agent: "Tôi tìm thấy vé đi Tokyo giá \$1,200."

FAILURE ANALYSIS

Nếu agent đề xuất chuyển bay giá \$1,200 đến Tokyo, nó đã **thất bại** trong thước đo độ mạch lạc.

Nó bị **"Suy giảm ngữ cảnh" (Context Decay)**.

PHÂN TÍCH KỸ THUẬT

- Context Decay xảy ra do hiện tượng **Pha loãng sự chú ý (Attention Dilution)** trong các mô hình Transformer.
- Khi cửa sổ trạng thái bị lấp đầy bởi các payload API JSON và quá trình suy luận scratchpad, cơ chế attention về mặt toán học sẽ **mất đi sự tập trung** vào các ràng buộc gốc.
- Chúng ta đánh giá bằng cách chèn các bài kiểm tra **"Mò kim đáy biển" (Needle in a Haystack)** vào lịch sử hội thoại và đo lường độ chính xác truy xuất ở Lượt 10 so với Lượt 2.

THÔNG SỐ KINH DOANH (ROI)

- Xét cho cùng, đánh giá agent quy về **giá trị thực tiễn doanh nghiệp (Business ROI)**.
- **Tỷ lệ Giải quyết Tác vụ (Task Resolution Rate):** Agent có chạm đến trạng thái kết thúc thành công (VD: đặt xong vé) mà không cần bàn giao lại cho nhân viên không?
- **Tỷ lệ Bỏ rơi (Session Abandonment Rate):** Người dùng tức giận và chủ động đóng cửa sổ chat.
- **Nguyên tắc chung:** Hiệu quả Quỹ đạo cao + Độ chính xác Payload cao = Tỷ lệ Giải quyết Tác vụ cao.

PHÂN TÍCH KỸ THUẬT

- Làm thế nào để tự động đánh giá **Tỷ lệ Giải quyết Tác vụ?**
- Chúng ta sử dụng một **Judge LLM** để thực hiện Dialog State Tracking (DST).
- Ở mỗi lượt, Judge trích xuất **"Trạng thái"** mục tiêu của người dùng.
- Chúng ta đánh giá xem trạng thái được theo dõi cuối cùng có khớp với **trạng thái kết thúc thành công** hay không.

ĐỒ HỌA SO SÁNH DST

U: Tôi muốn đi Tokyo.

A: Bạn có ngân sách bao nhiêu?

U: Khoảng 500 USD.

A: Đang tìm kiếm...



```
{  
  "destination": "Tokyo",  
  "budget": 500,  
  "status": "searching"  
}
```

The Science of LLM-as-a-Judge & SOTA Benchmarks

Treating the LLM as a scientific instrument: Quantifying inherent biases, implementing rigorous mitigation strategies, and exploring State-of-the-Art pairwise evaluation models.



4.1 The Anatomy of a Judge Prompt

Structuring the evaluator: Moving from open generation to a deterministic architecture by explicitly defining the System Persona, Context Block, and Task.

- Sử dụng chuyên gia con người để chấm điểm AI là rất chính xác nhưng không thể mở rộng về quy mô (**statistically unscalable**).
- Tiêu chuẩn ngành hiện nay là "**LLM-as-a-Judge**" — sử dụng các mô hình frontier (như GPT-4o) để đánh giá các mô hình khác.
- Thực tế: LLM là các giám khảo tiết kiệm chi phí, nhưng độ tin cậy của chúng có nhiều thiếu sót căn bản do các **thiên kiến tiềm ẩn (latent biases)**.
- **Mục tiêu:** Hãy coi LLM Judge như một công cụ khoa học. Chúng ta phải hiểu được biên độ sai số của nó, hiệu chuẩn nó và thiết kế kiến trúc để giảm thiểu thiên kiến.

Một Judge Prompt hoàn toàn khác với một Generation Prompt thông thường. Nó đòi hỏi một kiến trúc tất định (**deterministic**) cực kỳ nghiêm ngặt.

1. System Persona: Ép LLM đảm nhận vai trò của một giám khảo cứng rắn (VD: "Bạn là một chuyên gia đánh giá vô tư, khắt khe...").

2. Khối Ngữ cảnh (Context Block): Phân định rõ Truy vấn của Người dùng, Ngữ cảnh được Truy xuất và Câu trả lời của Agent.

3. Định nghĩa Tác vụ: Hướng dẫn rõ ràng về những gì đang được đánh giá (VD: "Chỉ đánh giá độ trung thực, bỏ qua giọng điệu và định dạng").

- Yêu cầu LLM "Đánh giá từ 1 đến 5" mà không có định nghĩa sẽ chắc chắn cho ra dữ liệu vô dụng.
- Bạn phải định nghĩa các điều kiện ranh giới chính xác cho từng con số một.

Ví dụ về Rubric:

Điểm 1: Câu trả lời trực tiếp mâu thuẫn với ngữ cảnh.

Điểm 3: Câu trả lời chính xác nhưng bỏ sót các ràng buộc quan trọng trong ngữ cảnh.

Điểm 5: Câu trả lời hoàn toàn trung thực, đầy đủ và không chứa bất kỳ ảo giác nào.

- Quy tắc tối quan trọng nhất của LLM-as-a-Judge: Điểm số phải là **token cuối cùng** được sinh ra.
- Nếu LLM ngay lập tức sinh ra con số "4", nó sẽ dành toàn bộ phần còn lại để biện minh cho điểm "4" đó (Thiên kiến Xác nhận).

Giải pháp: Ép buộc tạo khối lý luận

Khối `<thought_process>` hoặc **Rationale**:

LLM phải tự suy luận qua rubric từng bước một trước khi được phép xuất ra kết quả cuối cùng.

Kết quả: `<score>`

Điểm số được đưa ra sau khi đã cân nhắc toàn bộ các tiêu chí, đảm bảo tính khách quan và chính xác.

4.2 Quantifying LLM Biases

The flawed oracle: Mathematically quantifying the inherent cognitive biases of LLM judges, from verbosity and position bias to systemic self-preference

Nghiên cứu (Zheng et al., 2023; Gu et al., 2024) chứng minh rằng **LLM Judges sở hữu những thiên kiến nhận thức nghiêm trọng.**

Nếu bạn không xử lý những điều này trong pipeline đánh giá, điểm RAGAS và kết quả benchmark của bạn **hoàn toàn vô giá trị.**

Nghiên cứu phân loại chúng thành 4 nhóm chính:

**Thiên kiến
Vị trí**

**Thiên kiến
Dài dòng**

**Thiên kiến
Tự ưu ái**

**Điểm yếu
Định dạng
Thù địch**

- Khi LLM được yêu cầu so sánh Câu trả lời A và Câu trả lời B, nó có sự **thiên vị đáng kể** đối với câu trả lời xuất hiện trước trong context của prompt.
- Điều này là do cơ chế attention **đặt trọng số nặng** lên các token xuất hiện sớm.
- Nếu A và B có chất lượng ngang nhau, LLM sẽ trao tỷ lệ chiến thắng nhiều hơn cho A một cách **thiếu cân xứng**.

Câu trả lời A

(Xuất hiện trước)

65% Win Rate

Câu trả lời B

(Xuất hiện sau)

35% Win Rate

- LLMs thường có xu hướng **đánh đồng độ dài** với chất lượng và độ sâu sắc.
- Khi đối mặt với một câu trả lời ngắn gọn/chính xác và một câu trả lời dài dòng/lan man, một LLM Judge chưa qua hiệu chuẩn thường sẽ **cho điểm câu trả lời lan man cao hơn**.
- Điều này chủ động **trừng phạt các agent tối ưu hóa tốt** và cổ vũ cho các phản hồi AI "phình to" vô ích.

Câu trả lời Ngắn gọn

(Chính xác, tối ưu)



Câu trả lời Dài dòng

(Lan man, phình to)



- Các LLM có một **"phương ngữ" rất riêng biệt** (VD: việc ChatGPT hay dùng từ "Delve", "Crucial").
- Các nghiên cứu chỉ ra rằng khi GPT-4 làm giám khảo, nó **ưu ái các câu trả lời** do các model khác của OpenAI sinh ra hơn là Claude hoặc Llama, vì nó nhận ra phân phối dữ liệu huấn luyện ẩn.

Ghi chú quan trọng

Mô hình có xu hướng chấm điểm cao hơn cho các câu trả lời khớp với phong cách ngôn ngữ của chính mình.

- "Chỉ một token để đánh lừa LLM-as-a-Judge": Các bộ đánh giá **rất dễ bị hack**.
- Nếu câu trả lời bao gồm các bảng markdown, in đậm, hoặc "Let's think step by step", Judge LLM sẽ tự động **bơm điểm số lên cao** một cách giả tạo.
- Judge đã nhầm lẫn giữa **"định dạng đẹp"** với "độ chính xác về sự thật".



Cảnh báo Thiên kiến

Mô hình có xu hướng ưu tiên các phản hồi có cấu trúc **Markdown chuyên nghiệp** thay vì tập trung vào tính **Xác thực của dữ liệu**.

4.3 Mitigation Strategies

Calibrating the judge: Implementing swap-testing, few-shot anchoring, and ensemble voting to mathematically eliminate systemic bias from your evaluation pipeline.

Giải quyết vấn đề

Thiên kiến Vị trí (Position Bias). Bất cứ khi nào chạy một phép so sánh cặp (A vs B), bạn phải chạy cùng một prompt đó hai lần.

Quy trình thực hiện

Lần 1: Prompt [Context, A, B] → Kết quả 1

Lần 2: Prompt [Context, B, A] → Kết quả 2

Xử lý mâu thuẫn

Nếu Judge chọn A ở Lần 1, nhưng chọn B ở Lần 2, kết quả đó sẽ bị loại bỏ với lý do "*Hòa không nhất quán*" (*Inconsistent Tie*).

Giải quyết vấn đề

Thiên kiến **Dài dòng** và Thiên kiến **Định dạng**. Chấm điểm Zero-shot là rất yếu; bạn **phải** sử dụng prompt Few-Shot.

Cung cấp "Ví dụ Vàng"

- Cung cấp 2-3 ví dụ trong System prompt.
- Mẫu điểm 5: Câu trả lời ngắn gọn, súc tích.
- Mẫu điểm 2: Câu trả lời lan man, rỗng tuếch.

Cơ chế hoạt động

Thao tác này dùng toán học để "**neo**" các trọng số attention của Judge vào khía cạnh logic sự thật thay vì bị đánh lừa bởi số lượng từ vựng.

Giải quyết vấn đề

Thiên kiến **Tự ưu ái (Self-Preference Bias)**. Không bao giờ dựa vào một dòng mô hình duy nhất cho các benchmark quan trọng trên production.

Quy trình Ensembling

Xây dựng pipeline đánh giá sử dụng "**Hội đồng Chuyên gia**" (Panel of Experts).

- GPT-4o
- Claude 3.5 Sonnet
- Llama-3-70B

Cơ chế tổng hợp

Tổng hợp điểm số bằng phương pháp **bỏ phiếu theo số đông** (majority vote) để triệt tiêu các sở thích cá nhân của từng mô hình.

4.4 Pairwise Evaluation & Chatbot Arena

Beyond absolute scoring: Utilizing Pairwise comparisons and Elo ratings to build statistically rigorous leaderboards that remain stable across model updates.

Tính Không Ổn Định

Ngay cả khi có các chiến lược giảm thiểu, yêu cầu LLM đưa ra một điểm số tuyệt đối (1-5) là không ổn định.

Hiện Tượng "Trôi Điểm" (Score Drift)

Các đợt cập nhật mô hình gây ra hiện tượng trôi điểm. Một câu trả lời đạt 4.5 tháng trước có thể chỉ được 3.8 hôm nay dù dùng chung prompt.

Xu Hướng Công Nghiệp: Chuyển Dịch Sang Đánh Giá Cặp

Để đạt sự ổn định dài hạn, ngành công nghiệp đang chuyển dần từ **Chấm điểm Tuyệt đối** sang **Đánh Giá Cặp (Pairwise Evaluation)**.

Cơ Chế Đánh Giá

- Thay vì chấm điểm từ 1 đến 5, prompt chỉ đơn giản hỏi: "Câu nào tốt hơn: Câu trả lời A hay Câu trả lời B?"
- Đây là một nhiệm vụ nhận thức dễ dàng hơn đáng kể đối với LLM.

Ứng Dụng Thực Tế

Đây chính xác là phương pháp luận được sử dụng bởi LMSYS Chatbot Arena, crowdsourcing hàng ngàn trận đấu đối kháng để tìm ra đâu thực sự là các mô hình State-of-the-art.

Cơ Sở Lý Thuyết

- Các kết quả đánh giá cặp (Thắng, Thua, Hòa) được mô hình hóa toán học thông qua mô hình Bradley-Terry.
- Nó tính toán xác suất để Mô hình A đánh bại Mô hình B dựa trên "sức mạnh" tiềm ẩn (latent strength) của chúng.

Công Thức Toán Học

$$P(A \text{ beats } B) = \frac{e^{\text{strength}_A}}{e^{\text{strength}_A} + e^{\text{strength}_B}}$$

Ý Nghĩa Thực Tiễn

Điều này cho phép xây dựng một bảng xếp hạng toàn cầu duy nhất, cực kỳ chặt chẽ về mặt thống kê từ vô số trận A/B ngẫu nhiên.

Để cập nhật các bảng xếp hạng một cách linh hoạt, chúng ta sử dụng hệ thống đánh giá Elo.

Điểm Kỳ vọng (Expected Score)

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

Kết Luận

Đánh giá giờ đây trở thành một giải đấu liên tục thay vì bài kiểm tra tĩnh.

Cập nhật Đánh giá

$$R'_A = R_A + K \cdot (S_A - E_A)$$

- R_A : Xếp hạng hiện tại của Mô hình A.
- K: Hệ số K (Mức độ thay đổi điểm số).
- S_A : Kết quả (1: Thắng, 0.5: Hòa, 0: Thua).

4.5 Specialized Evaluators

The rise of purpose-built judges: Exploring why fine-tuned 8B models like Prometheus often outperform frontier models at evaluation tasks while slashing latency and cost.

(SOTA): Bộ Đánh Giá Chuyên Biệt

Vấn đề quy mô

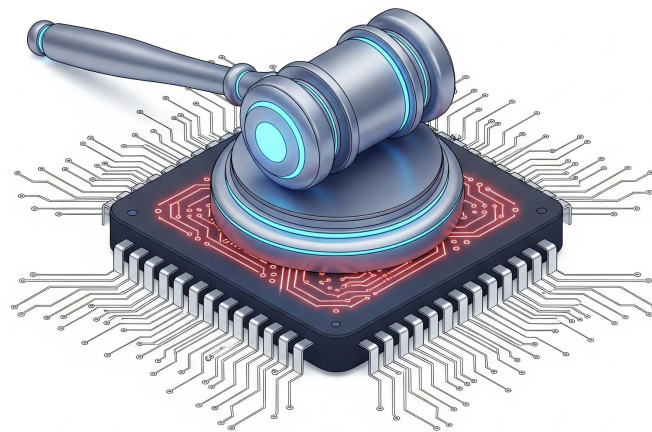
Chúng ta có thực sự cần một mô hình frontier 1.5 nghìn tỷ tham số để chấm điểm pipeline RAG đơn giản không? **Không.**

Hiệu quả chi phí

Sử dụng GPT-4o cho các bài đánh giá quy mô lớn là rất đắt đỏ và chậm chạp.

Sự dịch chuyển SOTA

Ngành công nghiệp đang hướng tới các mô hình nhỏ hơn (8B đến 70B), được tinh chỉnh độc quyền chỉ để đóng vai trò làm giám khảo.



Prometheus & JudgeLM

Mô hình Mã nguồn mở chuyên biệt

Các mô hình như Prometheus-2 và JudgeLM là mã nguồn mở được huấn luyện trên hàng triệu ví dụ feedback được con người chấm điểm.

Hiệu suất vượt trội (Prometheus 8B)

Bởi vì chúng được đào tạo chuyên biệt dựa trên rubric, một mô hình Prometheus 8B thường xuyên có thể sánh ngang hoặc vượt mức độ tương quan (κ) của GPT-4 so with human experts.

Lợi ích

Bạn có thể deploy chúng locally bên trong pipeline CI/CD, chạy hàng triệu lượt đánh giá chỉ với chi phí cực thấp.



Ranh Giới Đột Phá: Agent-as-a-Judge

Bước nhảy vọt tiếp theo

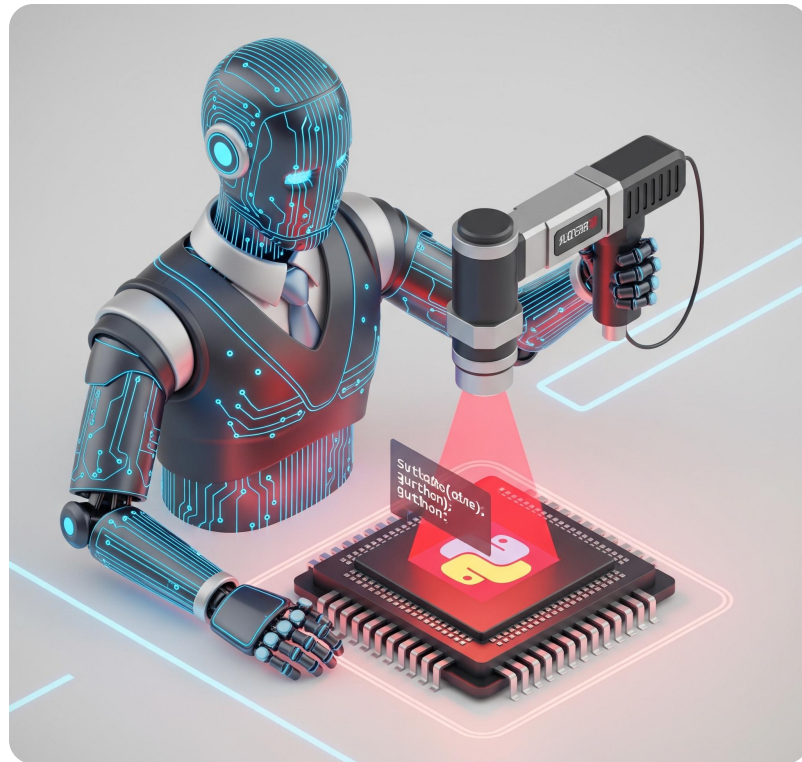
Vượt ra khỏi ranh giới của một prompt đơn lẻ để tiến tới khả năng đánh giá đa tầng.

Hạn chế của Judge LLM

Nếu được yêu cầu xác minh đoạn mã Python, nó có thể gặp ảo giác và cho rằng đoạn mã đó chạy được.

Giải pháp: Agent-as-a-Judge

Chúng ta cấp cho bộ đánh giá các Tools. Judge Agent sẽ tự động viết unit test, thực thi mã trong sandbox, đọc đầu ra terminal và chấm điểm dựa trên kết quả thực nghiệm.



HƯỚNG DẪN HOẠT ĐỘNG

- **Hình thức:** Làm việc cá nhân hoặc Theo cặp
- **Thời gian:** 30 - 45 phút
- **Nhiệm vụ của bạn:** Khai thác lỗ hổng nhận thức của LLM! Hãy thiết lập một kịch bản chấm điểm tự động, sau đó cố tình tạo ra dữ liệu để **kích hoạt một thiên kiến (bias)** khiến LLM-as-a-Judge đưa ra quyết định sai lầm.

Tài nguyên sử dụng:

- Sử dụng các API trả phí (OpenAI, Gemini, Anthropic) qua API key.
- **Khuyến nghị (Để "hack" hơn):** Tái sử dụng code/môi trường của **Lab Ngày 1** để chạy một mô hình mã nguồn mở nhỏ (Local Small Models như *Qwen-0.5B*, *Llama-3-8B*). Mô hình càng nhỏ, thiên kiến càng bộc lộ rõ rệt!

MLOps, Cost, and Continuous Improvement

The engineering of GenAI: Navigating the production trade-offs between accuracy, latency, and cost while building automated CI/CD pipelines for continuous agent improvement.

5

5.1 The Production Triangle

The Golden Ratio of LLMOps: Managing the inherent tension between model performance, inference speed, and token unit economics in high-scale applications.

Một điểm RAGAS cao trong Jupyter Notebook sẽ chẳng có ý nghĩa gì nếu hệ thống sụp đổ trước lưu lượng truy cập thực tế.

Thực tế Kỹ thuật

Việc đưa một AI Agent từ "Lab" lên "Production" đòi hỏi phải chuyển trọng tâm từ độ chính xác toán học thuần túy sang khả năng vận hành thực tiễn.

Mục tiêu

Xây dựng một pipeline LLMOps tự động, đối xử với các thay đổi prompt và cập nhật dữ liệu khắt khe như việc triển khai code phần mềm truyền thống.

Mỗi hệ thống AI production đều bị ràng buộc bởi 3 lực cản. Thông thường bạn chỉ có thể tối ưu cho 2, và yếu tố thứ 3 sẽ bị hy sinh.

1. Accuracy

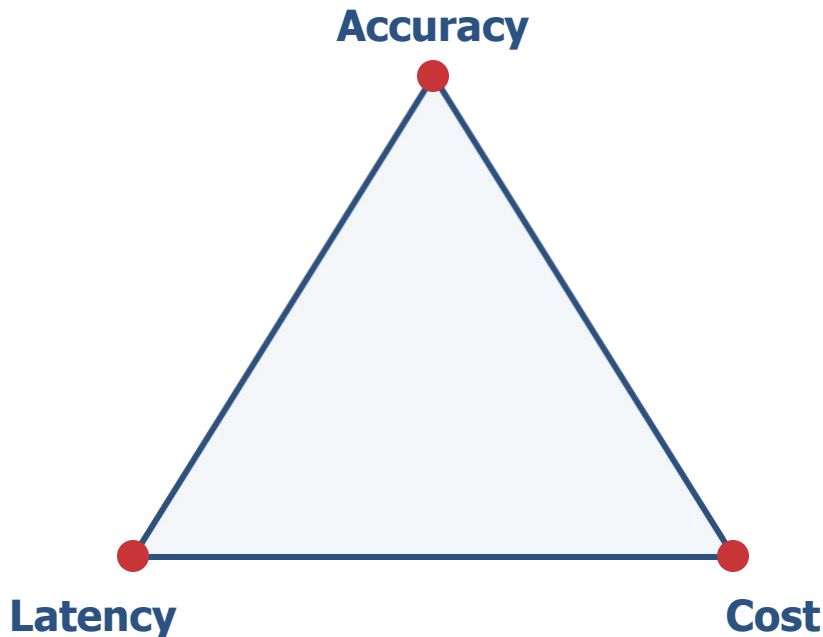
Tỷ lệ ảo giác, độ sâu suy luận, và tính đúng đắn sự thật.

2. Latency

Hệ thống phản hồi cho người dùng nhanh đến mức nào.

3. Cost

Tốc độ đốt token API, chi phí tính toán GPU, và hạ tầng.



Với tư cách là Kỹ sư, bạn phải chọn một kiến trúc phù hợp với SLA của Doanh nghiệp.

Kịch bản A

Accuracy ↑, Latency ↓, Cost ↑

Sử dụng GPT-4o cho mọi lượt trò chuyện với Context window khổng lồ. Trải nghiệm UX tuyệt vời nhưng sẽ làm doanh nghiệp phá sản.

Kịch bản B

Accuracy ↑, Cost ↓, Latency ↑

Định tuyến truy vấn phức tạp đến mô hình Llama-3-70B local chạy bằng hàng đợi. Rẻ và chính xác, nhưng người dùng phải đợi 15 giây.

Kịch bản C

Cost ↓, Latency ↓, Accuracy ~

Dùng một mô hình nhỏ, tốc độ cao (VD: Claude 3 Haiku) kèm với truy xuất BM25 nghiêm ngặt.

5.2 Latency Metrics

The UX of speed: Deconstructing AI latency into TTFT and TPS to measure the gap between raw model inference and the user's perception of real-time intelligence.

Trong phần mềm truyền thống, thời gian tải 2 giây là quá tệ. Trong Generative AI, việc sinh ra câu trả lời 500 từ có thể mất đến 10 giây.

Chiến thuật Streaming

Để ngăn chặn việc người dùng rời bỏ, chúng ta dùng tính năng Streaming (Phát luồng văn bản). Nhưng Streaming lại đòi hỏi thước đo riêng biệt.

Phân chia Giai đoạn

Chúng ta chia AI Latency thành hai giai đoạn: TTFT (Time to First Token) và TPS (Tokens Per Second).

Định nghĩa: Thời gian trôi qua tính từ lúc người dùng nhấn "Gửi" cho đến khi ký tự đầu tiên hiện lên màn hình.

Thành phần cấu tạo TTFT

TTFT bao gồm độ trễ mạng, thời gian tìm kiếm Vector DB, thời gian xử lý prompt của LLM (pre-fill), và những suy luận ban đầu.

Tại sao nó quan trọng?

TTFT quyết định cảm nhận về tốc độ. TTFT dưới 1.0 giây mang lại cảm giác tức thời, giữ người dùng tập trung ngay cả khi tin nhắn mất đến 5 giây để xuất ra trọn vẹn.

Định nghĩa: Khi luồng văn bản bắt đầu phát, các token tiếp theo được tạo ra với tốc độ bao nhiêu? (Throughput).

Mốc tham chiếu: Tốc độ đọc trung bình của con người rơi vào khoảng 4-5 từ (5-7 tokens) mỗi giây.

UX Tuyệt vời

Nếu TPS của bạn > 10 , AI sinh văn bản nhanh hơn mức người dùng có thể đọc.

Trải nghiệm Tệ

Nếu TPS của bạn < 3 , người dùng sẽ phải chờ AI viết xong câu, dẫn đến sự khó chịu tốt độ.

5.3 Evaluation in CI/CD

Automating the feedback loop: Integrating RAGAS and custom evaluators into GitHub Actions to ensure zero regressions in faithfulness or relevance during deployment.

Đánh giá không thể là một đoạn script chạy thủ công vào Thứ Sáu. Nó phải được tích hợp vào pipeline CI/CD (VD: GitHub Actions, GitLab CI).

Bộ kích hoạt (Trigger)

Mỗi khi một kỹ sư cập nhật System Prompt, bổ sung Tool mới, hoặc tinh chỉnh chiến lược chunking, một pull request sẽ được tạo ra.

Tự động hóa Kiểm thử

Pull request này phải tự động kích hoạt pipeline RAGAS / LLM-as-a-Judge đối chiếu với Golden Dataset.

Quality Gate là một ngưỡng tự động sẽ ngăn chặn việc triển khai lên production nếu các tiêu chuẩn không được đáp ứng.

Ví dụ về các Quy tắc

Context_Recall $\geq$ Baseline

Faithfulness > 0.85

Average_TTFT $< 1.5s$

Tự động Chặn (Build Fail)

Nếu prompt mới làm tăng Accuracy nhưng lại khiến chi phí Token tăng vọt 300% (vượt quá ngưỡng), bản build đó sẽ báo lỗi (fail).

Làm thế nào để kiểm thử một đợt nâng cấp mô hình lớn (VD: GPT-4 sang Claude 3.5) mà không gây rủi ro cho UX? Sử dụng **Dark Launching**.

Quy trình: Baseline

Định tuyến 100% lưu lượng truy cập của người dùng vào Agent cũ đang ổn định (**Mô hình Baseline**).

Người dùng sẽ chỉ nhìn thấy câu trả lời từ Mô hình Baseline.

Chạy ngầm (Shadow)

Chạy ngầm ở background, sao chép bất đồng bộ 10% trong số các truy vấn đó và gửi vào Agent Mới (**Mô hình Shadow**).

Hệ thống so sánh kết quả mà không ảnh hưởng tới trải nghiệm thực tế.

Khi bạn đã có cả đầu ra từ Baseline và đầu ra từ Shadow cho cùng một truy vấn thực tế, bạn sẽ đánh giá độ chênh lệch (Delta) bằng cách tính toán Cosine Similarity giữa hai câu trả lời đó.

Ngưỡng So Sánh (Metrics)

Similarity > 0.95

Similarity < 0.70

Hành Động Hệ Thống

- **An toàn:** Mô hình mới hoạt động giống hệt mô hình cũ, sẵn sàng triển khai.
- **Cảnh báo:** Phát hiện bất thường! Gắn cờ (flag) để kỹ sư SRE xem xét thủ công trước khi triển khai chính thức.

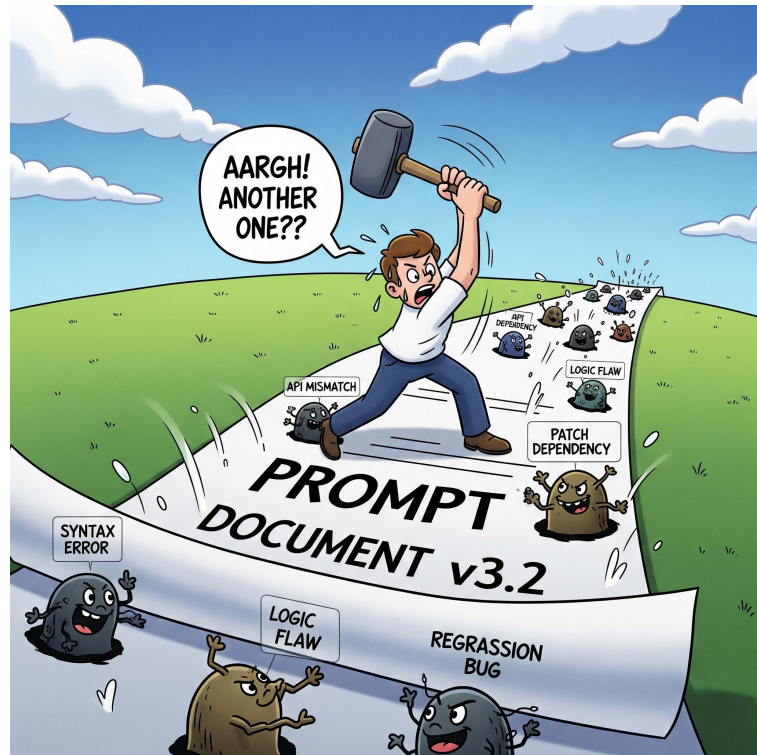
5.4 Failure Clustering

Systemic debugging at scale: Using K-Means on failed query embeddings to identify recurring failure modes and prioritize data pipeline fixes over prompt patching.

Khi xem xét các lượt đánh giá bị lỗi, kỹ sư thường rơi vào bẫy **"Vá Prompt"** (Prompt Patching).

- Người dùng: "Agent báo sai giờ mở cửa." → Kỹ sư: Thêm câu "Luôn kiểm tra kỹ giờ làm việc" vào prompt.
- Điều này khiến prompt ngày càng phình to, tăng chi phí token, và thường làm vỡ các tính năng khác (**Lỗi hồi quy - Regression**).

Giải pháp: Đừng sửa từng lỗi riêng lẻ. Hãy sửa các cụm lỗi mang tính hệ thống.



Failure Clustering: Phân Tích K-Means

Quy trình thực hiện:

- Thu thập tất cả các truy vấn trong tuần qua nhận điểm thấp về Answer Relevance hoặc Faithfulness.
- Cho các truy vấn lỗi này chạy qua một Embedding Model để chuyển chúng thành các vector.
- Áp dụng thuật toán K-Means Clustering để gom nhóm (cluster) các vector lỗi này lại với nhau trong không gian toán học.



Sửa Chữa Nguyên Nhân Gốc Rễ

Phân tích các cụm K-Means sẽ phơi bày các sự cố mang tính hệ thống.

- **Cụm 1 (45% lỗi):** Liên quan đến "Giá cả" → **Lỗi mã hóa PDF**
- **Cụm 2 (30% lỗi):** So sánh sản phẩm → **Chunk size quá nhỏ**

Chỉ cần sửa data pipeline là đã khắc phục ngay lập tức 75% toàn bộ lỗi mà không cần Prompt engineering.



Tạo benchmark cho agent, chạy evaluation, phân tích failures, và đề xuất improvements dựa trên data.

1. Tạo golden dataset: 20 question-answer pairs với expected answers
2. Chạy agent trên toàn bộ dataset, collect results
3. RAGAS evaluation: faithfulness, answer relevancy, context recall, context precision
4. LLM-as-Judge: scoring 1-5 với rubric cho 10+ responses
5. Failure analysis: chọn 3 worst cases, 5 Whys cho mỗi case
6. Improvement log: ghi lại recommendations dựa trên root cause

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Evaluation là engineering discipline**, không phải cảm tính. “Cảm thấy tốt” không phải evidence, RAGAS scores mới đáng tin cậy.
- 2 **LLM-as-Judge + RAGAS** = automated quality gate. Chạy mỗi release, block deploy nếu score dưới ngưỡng.
- 3 **Failure analysis** là fastest path to improvement. Cluster failures, tìm root cause, fix systematic thay vì từng case.
- 4 **Continuous improvement loop**: evaluate, analyze, improve, augment benchmark. Lặp lại mỗi sprint.

1. RAGAS Documentation — docs.ragas.io. Evaluate RAG: faithfulness, answer relevancy, context recall.
2. OpenAI Evals — github.com/openai/evals. Framework cho custom evaluation pipelines.
3. Zheng et al. (2023), Judging LLM-as-a-Judge — [arXiv:2306.05685](https://arxiv.org/abs/2306.05685). Bias types, rubric design, MT-Bench.

Hỏi & Đáp

Bạn đang thiếu model mạnh hơn, hay đang thiếu một pipeline retrieval và evaluation đủ kỷ luật?